

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 02 -

IMAGENS E CONTAINERS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS	12

EMSE

O QUE VEM POR AÍ?

Agora que a gente já passou pelos conceitos iniciais e entendeu o que é um container, como ele se diferencia de uma máquina virtual e qual a lógica por trás da arquitetura que o Docker utiliza, chegou a hora da parte prática.

Nesta aula, vamos falar sobre imagens Docker: o que elas são, como se relacionam com containers e qual o papel delas no dia a dia de quem desenvolve ou opera aplicações. Vamos explorar o Docker Hub, conhecer alguns comandos fundamentais como `dockerpull`, `dockerrun`, `dockerps` e `docker stop`, e entender como lidar com o ciclo de vida de containers — desde o momento em que você baixa uma imagem até a hora de remover ou recriar um container.

HANDS ON

Chegamos na parte prática e agora é a hora de começar a fazer. A ideia aqui é pegar todo o conteúdo e ver como isso se traduz na vida real, no terminal, onde a coisa acontece de verdade.

No primeiro vídeo, a gente vai bater um papo sobre o que é uma imagem Docker — com calma, sem mistério. Você vai entender qual é a diferença entre imagem e container, por que isso importa e como essa estrutura toda facilita a nossa vida no desenvolvimento. Temos uma analogia com receita e prato pronto que pode parecer boba, mas ajuda bastante a visualizar.

O segundo vídeo, é mão na massa. Terminal aberto e comandos direto ao ponto: como encontrar imagens no Docker Hub, como baixar, rodar, parar e remover containers. Vamos usar o Nginx como exemplo e fazer tudo isso junto. O objetivo aqui é te deixar confortável com o básico que você vai usar todo dia.

No terceiro vídeo, a gente sobe um nível. Mostraremos como criar imagens mais limpas, mais seguras e preparadas pra produção. Aí entra o “multistage build”, além de umas dicas para evitar uma imagem gigante e também como identificar o que “deu ruim” quando o build quebra. Aqui teremos o suficiente pra você sair com uma base sólida e saber por onde seguir.

Se quiser, pode ir abrir o terminal e testar junto comigo!

SAIBA MAIS

Docker Hub e repositórios de imagens

Docker Hub é o repositório oficial e mais utilizado para armazenar, distribuir e gerenciar imagens Docker. É como se fosse um "GitHub das imagens Docker", feito para simplificar. O Docker Hub permite que você encontre facilmente imagens prontas para uso, como o Nginx, o Node.js, ou até mesmo sistemas operacionais inteiros, como Ubuntu e Alpine.

Essas imagens podem ser organizadas em dois tipos principais de repositórios:

Repositórios públicos: são acessíveis a qualquer pessoa. Não precisa nem fazer login para baixar uma imagem pública do Docker Hub. Por exemplo, se você roda o comando:

```
dockerpullnginx
```

automaticamente você baixa a imagem oficial e pública do Nginx que está disponível no Docker Hub.

Os repositórios públicos são ótimos para distribuir software open-source ou imagens oficiais mantidas por grandes empresas ou comunidades.

Repositórios privados: aqui, o acesso é controlado. Normalmente são usados por empresas ou indivíduos que não querem expor suas imagens ao público geral. Para baixar imagens privadas você precisa se autenticar no Docker Hub, usando o comando:

```
docker login
```

Depois disso, é só rodar:

```
dockerpull nome-da-imagem
```

Essa abordagem permite que times e organizações controlem exatamente quem acessa suas imagens, garantindo segurança e privacidade.

E como empresas maiores lidam com isso?

Em ambientes corporativos, o cenário é um pouco diferente. Muitas empresas grandes preferem hospedar seus próprios repositórios privados dentro da infraestrutura interna ou em serviços de cloud específicos, como:

- AWS Elastic Container Registry (ECR);
- Azure Container Registry (ACR);
- Google Container Registry (GCR);
- JFrog Artifactory ou Harbor (soluções open-source e auto-hospedadas).

Essas ferramentas permitem maior controle sobre segurança, compliance e armazenamento. Além disso, elas costumam se integrar diretamente com pipelines de CI/CD, automatizando a criação, distribuição e atualização das imagens internas.

Na prática, uma empresa pode ter uma estratégia híbrida:

- Imagens comuns (como bases Alpine ou Ubuntu) são buscadas no Docker Hub público;
- Imagens sensíveis, que contêm o código da aplicação proprietária, ficam guardadas em repositórios internos, acessíveis apenas pela equipe autorizada.

Pontos para sempre lembrar:

- ✓ Docker Hub é o ponto inicial para descobrir e utilizar imagens públicas;
- ✓ Use imagens públicas sempre que possível, desde que confiáveis e verificadas;
- ✓ Use imagens privadas para código sensível ou interno, controlando o acesso cuidadosamente;
- ✓ Em ambientes maiores, considere soluções internas ou de terceiros para melhor segurança e controle.

Se precisar voltar aqui, lembre-se: esses conceitos não mudam muito, mas as ferramentas e boas práticas evoluem constantemente. Vale sempre revisar periodicamente e ficar por dentro das novidades!

Dockerfile (FROM, RUN, CMD, COPY, WORKDIR)

Se tem uma coisa que você vai escrever muito ao trabalhar com Docker, é o tal do Dockerfile. Ele é só um arquivo de texto, mas com muito poder: é ali que você define o passo a passo para construir a imagem que vai rodar sua aplicação. Funciona como um script declarativo — você diz o que quer que aconteça e o Docker constrói.

O processo é bem linear, e com o tempo você decora a ordem das instruções. Mas no começo, vale entender o papel de cada uma.

FROM

Tudo começa com o FROM. É aqui que você escolhe a base da sua imagem. Pode ser o Node, o Python, o Ubuntu ou qualquer outra coisa. O mais comum é usar imagens oficiais que já vêm prontas no Docker Hub.

Exemplo: FROM node:20

Isso quer dizer: “pegue a imagem do Node na versão 20 e comece a construir em cima dela”.

WORKDIR

Depois disso, é comum definir um diretório de trabalho com WORKDIR. Isso nada mais é do que dizer: “a partir de agora, tudo que eu fizer vai ser dentro desta pasta aqui”. Isso evita que você tenha que dar comandos com caminhos longos o tempo todo.

Exemplo: WORKDIR /app

COPY

Aí vem o COPY, que serve para colocar seus arquivos dentro da imagem. Ele seleciona o que está em sua máquina e joga dentro da imagem que está sendo construída.

Exemplo: COPY . /app

Isso copia tudo o que está no diretório atual para o /app da imagem.

RUN

Com os arquivos no lugar, você pode executar comandos com RUN. Ele é muito usado para instalar dependências, criar pastas, mover arquivos, o que for necessário durante a construção da imagem.

Exemplo: RUN npm install

Esse comando vai rodar dentro da imagem e o resultado fica salvo como uma camada nova.

CMD

Por fim, vem o CMD, que define o comando que vai rodar quando o container for iniciado. É como dizer: “quando alguém rodar essa imagem, execute isso aqui por padrão”.

Exemplo: CMD ["node", "index.js"]

Isso vai rodar o index.js usando o Node quando o container subir.

A ordem dessas instruções importa, principalmente porque o Docker guarda o cache de cada etapa. Se você mudar algo no topo do Dockerfile, ele vai reconstruir tudo abaixo. Agora, se você só mexer no CMD, por exemplo, ele reaproveitará o restante.

Com o tempo você vai aprender a montar Dockerfiles mais limpos, diretos e fáceis de manter. Mas comece entendendo bem o básico: que o Dockerfile é onde você escreve a lógica de construção da sua imagem. Sem mágica, sem mistério. Só prática e clareza.

Imagens base e minimalismo

Se você já viu imagens com nomes do tipo node:alpine ou python:3.11-slim e ficou se perguntando o que muda, a resposta é: muita coisa. Essas versões são conhecidas como imagens enxutas — ou minimalistas. Elas são menores, ocupam menos espaço em disco e, em geral, são mais rápidas pra transferir e subir. E para quem trabalha com deploy, isso pesa.

Mas calma, não é só vantagem. Quando você escolhe uma imagem enxuta, você também abre mão de várias coisas que vêm “de fábrica” nas imagens maiores. A alpine, por exemplo, é uma distribuição Linux mínima, com menos de 10MB. Bem leve, mas também bem limitada. Muita coisa que você acha que “vem por padrão”,

por exemplo bash, curl, ou até ferramentas básicas de build, simplesmente não estão lá. E aí você vai ter que instalar manualmente com apkadd (o gerenciador de pacotes do Alpine).

Já as versões slim seguem a mesma ideia: removem o que não é essencial, mas sem ir tão longe quanto o alpine. Costumam funcionar melhor quando você precisa de um meio-termo — menos peso, mas ainda com alguma estrutura útil.

Então, quando usar? Se você tá preparando uma imagem para ambiente de produção e já sabe exatamente o que vai rodar dentro do container, usar alpine ou slim pode ser uma boa. Isso reduz o tamanho final da imagem e diminui a superfície de ataque, o que também ajuda em segurança.

Agora, para ambiente de desenvolvimento ou quando você ainda está montando o Dockerfile e não sabe o que vai precisar instalar, talvez seja melhor começar com uma imagem mais completa. Depois que tudo estiver funcionando, aí sim você pode tentar “secar” a imagem.

No fim, a escolha da imagem base tem a ver com equilíbrio. O mais leve nem sempre é melhor. Comece com o que te dá menos “dor de cabeça”, e vá otimizando depois. Sem pressa, sem sofrimento.

Multistage Builds

Sabe quando você precisa instalar um monte de coisa só para compilar ou construir sua aplicação, mas depois que ela está pronta, nada disso precisa estar no ambiente final? Pois é, o Multistage Build resolve exatamente isso.

A ideia é simples: você usa uma imagem cheia de ferramentas somente durante a fase de build, e depois copia apenas o que interessa para uma imagem mais limpa, que vai ser usada na hora de rodar a aplicação. Isso ajuda a manter a sua imagem final menor, mais segura e sem “tralha” desnecessária.

Na prática, funciona assim: você define uma etapa FROM para build, faz o que precisa (instala dependência, roda build, compila, etc.), e depois abre um novo FROM limpo, copiando só os arquivos que você precisa executar de fato.

Por exemplo, no caso de uma app Node.js:

```
# Etapa 1 - build
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install &&npm run build

# Etapa 2 - produção
FROM node:20-slim
WORKDIR /app
COPY --from=build /app/dist ./dist
CMD ["node", "dist/index.js"]
```

Comando de Prompt 1 – Etapa FROM para build

Fonte: elaborado pelo autor (2025)

A primeira etapa instala tudo, roda o build, e a segunda só carrega o resultado. Nada de node_modules, ferramentas de build ou arquivos temporários indo parar na imagem final.

Esse padrão é bem comum em projetos mais maduros, porque ajuda a deixar o deploy mais leve e evita que coisas desnecessárias (ou até perigosas) acabem indo parar em produção.

Se você ainda não usa, comece a testar em projetos pequenos. Depois que você se acostumar, vira um padrão.

O QUE VOCÊ VIU NESTA AULA?

Ao longo dessa aula, a gente saiu do conceito e foi para a prática. Começamos entendendo o que é uma imagem Docker, como ela se relaciona com um container e por que isso é importante. Discutimos a diferença entre imagem e container — uma é o modelo, a outra é a execução — e reforçamos isso com exemplos e analogias que ajudam a visualizar o que acontece de verdade por trás do docker run.

Se você ficou alguma dúvida, volte aos vídeos, leia os materiais de apoio com calma e tente repetir os comandos por conta própria. E se você já está confortável com o que rolou aqui, ótimo! Na próxima aula, seguiremos nos aprofundando.

REFERÊNCIAS

DOCKER INC. **Dockerfile**reference. 2024. Disponível em: <https://docs.docker.com/reference/dockerfile/>. Acesso em: 27 maio 2025.

MERKEL, D. **Docker: lightweight Linux containers for consistent development and deployment**. Linux Journal, n. 239, 2014. Disponível em: <https://linuxjournal.com/article/8391>. Acesso em: 27 maio 2025.

TURNBULL, J. **The Docker book: containerization is the new virtualization**. [S.l.]: James Turnbull, 2014.

PALAVRAS-CHAVE

Docker. Imagem. Container.

EMSE



POSTECH